



Exp :11 Write the python program for Map Coloring to implement CSP.

Aim:

To write a Python program to solve the Map Coloring problem using the Constraint Satisfaction Problem (CSP) approach with backtracking.

Algorithm:

Step 1: Define the map as a graph using an adjacency list where keys are regions (variables) and values are lists of neighboring regions.

Step 2: Define a list of available colors (domains) that can be assigned to each region.

Step 3: Define a function `is_safe` to check if a color assignment is valid by verifying that no adjacent neighbor of the current region has the same color.

Step 4: Define a recursive backtracking function `solve_map_coloring` that takes the graph, colors, a dictionary of assignments, and a list of unassigned regions.

Step 5: If all regions are assigned a color, return True as the problem is solved.

Step 6: Select the first unassigned region from the list.

Step 7: Loop through each color in the domain and check if it is safe to assign using the `is_safe` function.

Step 8: If a color is safe, assign it to the current region and recursively call `solve_map_coloring` for the remaining unassigned regions.

Step 9: If the recursive call returns True, stop and return True.

Step 10: If assigning the color does not lead to a solution, backtrack by removing the color assignment and try the next color.

Step 11: If no color works for the current region, return False.

Step 12: In the main section, initialize the map graph, colors, and an empty assignment dictionary, then print the final coloring.

Program:

```
def is_safe(node, color, graph, assignment):  
    for neighbor in graph[node]:  
        if neighbor in assignment and assignment[neighbor] == color:  
            return False
```

```
    return True
```

```
def solve_map_coloring(graph, colors, assignment, nodes):
```

```
    if len(assignment) == len(graph):
```

```
        return True
```

```
    node = nodes[0]
```

```
    for color in colors:
```

```
        if is_safe(node, color, graph, assignment):
```

```
            assignment[node] = color
```

```
            if solve_map_coloring(graph, colors, assignment, nodes[1:]):
```

```
                return True
```

```
            del assignment[node]
```

```
    return False
```

```
graph = {
```

```
    'WA': ['NT', 'SA'],
```

```
    'NT': ['WA', 'SA', 'Q'],
```

```
    'SA': ['WA', 'NT', 'Q', 'NSW', 'V'],
```

```
    'Q': ['NT', 'SA', 'NSW'],
```

```
    'NSW': ['Q', 'SA', 'V'],
```

```
    'V': ['SA', 'NSW'],
```

```
    'T': []
```

```
}
```

```
colors = ['Red', 'Green', 'Blue']
```

```
assignment = {}
```

```
nodes = list(graph.keys())

if solve_map_coloring(graph, colors, assignment, nodes):
    print("Map Coloring Solution:")
    for region, color in assignment.items():
        print(f"{region} : {color}")
else:
    print("No solution exists!")
```

OUTPUT:

```
Map Coloring Solution:
WA : Red
NT : Green
SA : Blue
Q : Red
NSW : Green
V : Red
T : Red
```

Result: Thus, the program was executed and found that the output was found to be correct.